Xi'an Jiaotong-Liverpool University
西交利物浦大學

Module Code	Examiner	Department	Tel
INT104		INT	

2nd SEMESTER 24-25 FINAL EXAMINATION

Undergraduate

Artificial Intelligence

TIME ALLOWED: *2 hours*

INSTRUCTIONS TO CANDIDATES

1. This is an open-book exam and the duration is 2 hours.
2. Total marks available are 100. This accounts for 60% of the final mark.
3. Answer all questions. Relevant and clear steps should be included in the answers.
4. Please use MCQ card delivered to answer MCQ questions. Please use answer booklet for answer other questions.
5. Only English solutions are accepted.
6. The use of calculator is allowed.
7. All paper based materials are allowed. NO DICTIONARIES.

Section 2 Computation Questions (28 Marks)

19. Consider the following dataset with five data points in a 2D space:

$$A(1,2), B(2,2), C(5,3), D(6,4), E(7,5) \quad (1)$$

Perform hierarchical clustering using the agglomerative approach with Single linkage (minimum distance). For each method:

- Compute the distance matrices at each step.
- Draw the corresponding dendrogram.
- State the clusters formed when the number of clusters $k = 2$.

Please use **city block distance** to calculate the distance between samples and clusters i.e., $D(\alpha, \beta) = |\alpha_x - \beta_x| + |\alpha_y - \beta_y|$

(14 Marks)

20. Classify a new customer (**Age = Young**, **Income = Medium**) and predict why the customer will buy the product (i.e. **Buy=Yes/No**) using the training data:

Customer	Age	Income	Buy
1	Young	High	No
2	Young	High	No
3	Middle	High	Yes
4	Senior	Medium	Yes
5	Senior	Low	Yes
6	Senior	Low	No
7	Middle	Low	Yes
8	Young	Medium	No
9	Young	Low	Yes
10	Senior	Medium	Yes

(14 Marks)

Section 3 Programming Questions (18 Marks)

21. The following Python script classifies synthetic data using ensemble learning and cross-validation. Fill in the **five blanks** (labelled [**001**] to [**005**]) to complete the script correctly.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import KFold, train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Generate synthetic data
X, y = make_classification(n_samples=1000, n_features=10, random_state=42)

# Split data
[001], X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Initialize ensemble model
model = RandomForestClassifier([002]=50, random_state=42)

# Cross-validation setup
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = [003]
for train_idx, val_idx in kf.split(X_train):
    X_cv_train, X_cv_val = X_train[train_idx], X_train[val_idx]
    y_cv_train, y_cv_val = y_train[train_idx], y_train[val_idx]
    model.fit(X_cv_train, y_cv_train)
    val_pred = model.predict(X_cv_val)
    cv_scores.append(accuracy_score(y_cv_val, val_pred))

# Train on full training data
model.fit(X_train, y_train)

# Predict test set
test_pred = [004](X_test)
print(f"Test Accuracy: {[005](test_pred, y_test):.2f}%")
```

(10 Marks)

22. The following Python script classifies synthetic data using ensemble learning and cross-validation. Fill in the **five blanks** (labelled [**006**] to [**009**]) to complete the script correctly.

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Assume X is a pre-generated feature matrix (not shown)
X = np.loadtxt("clustering_data.csv")

# Preprocess data
scaler = StandardScaler()
X_scaled = [006].fit_transform(X)

# Initialize clustering model
model = KMeans(n_clusters=3, random_state=42)
clusters = model.fit_predict([007])

# Evaluate clustering performance
score = silhouette_score(X_scaled, [008])
print(f"Silhouette Score: {[009]}")
```

(8 Marks)

Section 4 Appendix: Useful API

API: `make_classification` (scikit-learn)

Function Signature

```
sklearn.datasets.make_classification(
n_samples=100,                  % Number of data points
n_features=20,                  % Total number of features
n_informative=2,                % Informative (relevant) features
n_redundant=2,                  % Redundant (linear combinations) features
n_classes=2,                    % Number of classes
random_state=None               % Seed for reproducibility
)
```

Key Parameters

- `n_samples`: Total number of data points (default: 100).
- `n_features`: Total number of features (default: 20).
- `n_informative`: Number of features useful for classification.
- `n_redundant`: Number of features linearly dependent on informative features.
- `n_classes`: Number of classes/labels (default: 2).
- `random_state`: Seed for random number generation (optional).

Returns

- `X`: Feature matrix of shape `(n_samples, n_features)`.
- `y`: Target labels of shape `(n_samples,)`.

API: `train_test_split` (scikit-learn)

Function Signature

```
sklearn.model_selection.train_test_split(
*arrays,                    % Input arrays (e.g., X, y)
test_size=None,             % Test set size (float or int)
train_size=None,            % Training set size (float or int)
random_state=None,          % Seed for reproducibility
shuffle=True,               # Whether to shuffle data
stratify=None               # Stratify split based on labels
)
```

Key Parameters

- `*arrays`: Input arrays to split (e.g., features `X`, labels `y`).
- `test_size`: Proportion (0.0–1.0) or absolute number of test samples. Default: `0.25`.
- `train_size`: Complement of `test_size` if not specified.
- `random_state`: Seed for shuffling (e.g., `42` for reproducibility).
- `shuffle`: Whether to shuffle data before splitting (default: `True`).
- `stratify`: Array-like object (e.g., labels `y`) for stratified splitting.

Returns

- List of split arrays: `[X_train, X_test, y_train, y_test, ...]`.
- Number of outputs matches the number of input arrays (always in train-test order).

API: `RandomForestClassifier` (scikit-learn)

Class Signature

```
sklearn.ensemble.RandomForestClassifier(
n_estimators=100,           % Number of trees in the forest
criterion='gini',           % Splitting criterion
max_depth=None,             # Maximum tree depth
min_samples_split=2,        % Min samples to split node
min_samples_leaf=1,         % Min samples at leaf node
random_state=None,          % Seed for reproducibility
class_weight=None           # Handle class imbalance
)
```

Key Parameters

- `n_estimators`: Number of decision trees in the forest (default: 100).
- `criterion`: Splitting criterion - `'gini'` (Gini impurity) or `'entropy'` (information gain).
- `max_depth`: Maximum depth of trees (default: None, expand until leaves are pure).
- `min_samples_split`: Minimum samples required to split a node (default: 2).
- `min_samples_leaf`: Minimum samples required at a leaf node (default: 1).
- `random_state`: Seed for random number generation (e.g., `42` for reproducibility).
- `class_weight`: Weights for classes (e.g., `'balanced'` for imbalanced data).

Key Methods

- `fit(X, y)`: Train the model on feature matrix `X` and labels `y`.
- `predict(X)`: Predict class labels for samples in `X`.
- `predict_proba(X)`: Return class probability estimates.
- `score(X, y)`: Return mean accuracy on test data `X` and labels `y`.

API: `KFold` (scikit-learn)

Class Signature

```
sklearn.model_selection.KFold(
n_splits=5,                    % Number of folds
shuffle=False,                 % Shuffle data before splitting
random_state=None              % Seed for reproducibility
)
```

Key Parameters

- `n_splits`: Number of folds (default: 5). Must be ≥ 2.
- `shuffle`: Whether to shuffle data before splitting (default: False).
- `random_state`: Seed for random number generation (required if `shuffle=True`).

Key Methods

- `split(X, y=None, groups=None)`: Generates indices to split data into training/test sets.
  - `X`: Feature matrix
  - Returns: `train_indices`, `test_indices` for each fold
- `get_n_splits(X=None, y=None, groups=None)`: Returns the number of splits (equal to `n_splits`).

API: `accuracy_score` (scikit-learn)

Function Signature

```
sklearn.metrics.accuracy_score(
y_true,              % Ground truth labels
y_pred,              % Predicted labels
normalize=True,      % Return accuracy (True) or count (False)
sample_weight=None   % Sample weights
)
```

Key Parameters

- `y_true`: Array-like of true labels (shape: [n_samples])
- `y_pred`: Array-like of predicted labels (shape: [n_samples])
- `normalize`:
  - `True` (default): Return fraction of correct predictions
  - `False`: Return number of correct predictions
- `sample_weight`: Array-like of weights for samples (optional)

Returns

- Float between 0 and 1 if `normalize=True` (accuracy percentage)
- Integer count of correct predictions if `normalize=False`

API: `StandardScaler` (scikit-learn)

Class Signature

```
sklearn.preprocessing.StandardScaler(
copy=True,          % Copy input data instead of modifying in-place
with_mean=True,     % Center data to mean=0
with_std=True       % Scale data to variance=1
)
```

Key Parameters

- `copy`: If `False`, try to avoid copying and normalize in-place (default: `True`)
- `with_mean`: Center the data before scaling (default: `True`)
- `with_std`: Scale data to unit variance (default: `True`)

Key Methods

- `fit(X)`: Compute mean and std for later scaling
- `transform(X)`: Perform standardization by centering and scaling
- `fit_transform(X)`: Fit to data, then transform it
- `inverse_transform(X)`: Scale back data to original representation
- `get_feature_names_out()`: Get output feature names

Attributes

- `mean_`: The mean value for each feature (computed during `fit`)
- `scale_`: The scaling factor (std deviation) for each feature
- `var_`: The variance for each feature

API: KMeans (scikit-learn)

Class Signature

```
sklearn.cluster.KMeans(
n_clusters=8,           % Number of clusters
init='k-means++',       % Initialization method
n_init=10,              % Number of initializations
max_iter=300,           % Maximum iterations per run
tol=1e-4,               % Convergence tolerance
random_state=None       % Seed for reproducibility
)
```

Key Parameters

- `n_clusters`: Number of clusters to form (default: 8)
- `init`: Initialization method:
  - `'k-means++'` (default): Smart initialization
  - `'random'`: Random initialization
  - Array: Custom initial cluster centers
- `n_init`: Number of random initializations (default: 10)
- `max_iter`: Maximum iterations per initialization (default: 300)
- `tol`: Relative tolerance for convergence (default: 1e-4)
- `random_state`: Seed for random number generation

Key Methods

- `fit(X)`: Compute clustering on data matrix X
- `predict(X)`: Predict cluster labels for X
- `fit_predict(X)`: Compute clustering and return labels
- `transform(X)`: Transform X to cluster-distance space
- `score(X)`: Negative inertia value (higher is better)

Attributes

- `cluster_centers_`: Coordinates of cluster centers (shape: [n_clusters, n_features])
- `labels_`: Cluster labels for training data
- `inertia_`: Sum of squared distances to nearest cluster center
- `n_iter_`: Number of iterations run for best initialization

API: `silhouette_score` (scikit-learn)

Function Signature

```
sklearn.metrics.silhouette_score(
X,                      % Feature array or distance matrix
labels,                 # Cluster labels
metric='euclidean',     % Distance metric
sample_size=None,       % Subsample for large datasets
random_state=None,      % Reproducibility seed
**kwargs                % Metric-specific parameters
)
```

Key Parameters

- `X`: Feature matrix (n_samples × n_features) or distance matrix if `metric='precomputed'`
- `labels`: Array of cluster labels (shape: [n_samples])
- `metric`: Distance metric (default: 'euclidean'), can be:
  - Predefined metrics: 'cosine', 'manhattan', etc.
  - 'precomputed' if X is a distance matrix
- `sample_size`: Subsample dataset for faster computation (default: None)
- `random_state`: Seed for random subsampling (required if sample_size ≠ None)

Returns

- Float: Mean Silhouette Coefficient for all samples
- Range: [-1, 1], where:
  - 1: Perfect clustering
  - 0: Overlapping clusters
  - -1: Incorrect clustering

END OF EXAM PAPER
THIS PAPER MUST NOT BE REMOVED FROM THE EXAMINATION ROOM